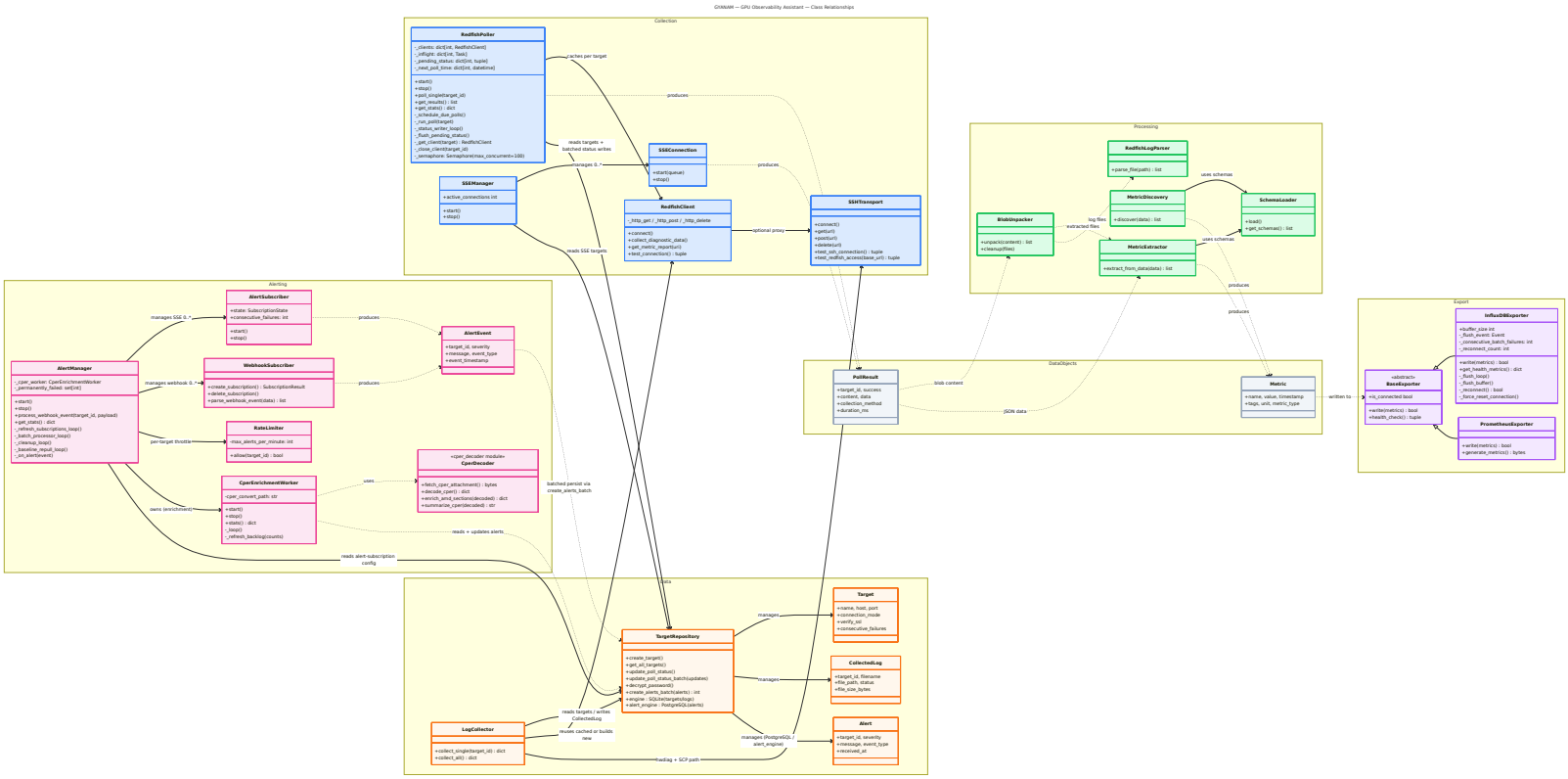

GHNAM — GPU Observability Assistant — Class Relationships
Collection
RedfishPoller
- _clients: dict[int, RedfishClient]
- _inflight: dict[int, Task]
- _pending_status: dict[int, tuple]
- _next_poll_time: dict[int, datetime]
+start()
+stop()
+poll_single(target_id)
+get_results() : list
+get_stats() : dict
- _schedule_due_polls()
- _run_poll(target)
- _status_writer_loop()
- _flush_pending_status()
- _get_client(target) : RedfishClient
- _close_client(target_id)
- _semaphore: Semaphore(max_concurrent=100)
creates per target
produces
reads targets + batched status writes
SSEConnection
+start(queue)
+stop()
SSEManager
+active_connections: int
+start()
+stop()
manages 0..*
produces
RedfishClient
- _http_get / _http_post / _http_delete
+connect()
+collect_diagnostic_data()
+get_metric_report(uri)
+test_connection() : tuple
optional proxy
SSHTransport
+connect()
+get(url)
+post(url)
+delete(url)
+test_ssh_connection() : tuple
+test_redfish_access(base_url) : tuple
reads SSE targets
Processing
RedfishLogParser
+parse_file(path) : list
uses schemas
BlobUnpacker
+unpack(content) : list
+cleanup(files)
log files extracted files
MetricDiscovery
+discover(data) : list
SchemaLoader
+load()
+get_schemas() : list
uses schemas
MetricExtractor
+extract_from_data(data) : list
produces
produces
Alerting
AlertSubscriber
+state: SubscriptionState
+consecutive_failures: int
+start()
+stop()
manages SSE 0..*
produces
AlertManager
- _cper_worker: CperEnrichmentWorker
- _permanently_failed: set[int]
+start()
+stop()
+process_redfish_event(target_id, payload)
+get_stats() : dict
- _refresh_subscriptions_loop()
- _batch_processor_loop()
- _cleanup_loop()
- _baseline_repull_loop()
- _on_alert(event)
WebhookSubscriber
+create_subscription() : SubscriptionResult
+delete_subscription()
+parse_webhook_event(data) : list
manages webhook 0..*
produces
AlertEvent
+target_id: severity
+message, event_type
+event_timestamp
RateLimiter
- max_alerts_per_minute: int
+allow(target_id) : bool
per-target throttle
CperEnrichmentWorker
-cper_convert_path: str
+start()
+stop()
+stats() : dict
- _loop()
- _refresh_backlog(counts)
owns (enrichment)
uses
«cper_decoder module»
CperDecoder
+fetch_cper_attachment() : bytes
+decode_cper() : dict
+enrich_amd_sections(decoded) : dict
+summarize_cper(decoded) : str
reads + updates alerts
reads alert-subscription config
batched persist via create_alerts_batch
DataObjects
PollResult
+target_id, success
+content, data
+collection_method
+duration_ms
blob content
JSON data
Metric
+name, value, timestamp
+tags, unit, metric_type
written to
Export
«abstract»
BaseExporter
#is_connected: bool
+write(metrics) : bool
+health_check() : tuple
InfluxDBExporter
+buffer_size: int
- _flush_event: Event
- _consecutive_batch_failures: int
- _reconnect_count: int
+write(metrics) : bool
+get_health_metrics() : dict
- _flush_loop()
- _flush_buffer()
- _reconnect() : bool
- _force_reset_connection()
PrometheusExporter
+write(metrics) : bool
+generate_metrics() : bytes
Data
TargetRepository
+create_target()
+get_all_targets()
+update_poll_status()
+update_poll_status_batch(updates)
+decrypt_password()
+create_alerts_batch(alerts) : int
+engine : SQLite(targets.db)
+alert_engine : PostgreSQL(alerts)
manages
Target
+name, host, port
+connection_mode
+verify_ssl
+consecutive_failures
manages
CollectedLog
+target_id, filename
+file_path, status
+file_size_bytes
manages (PostgreSQL alert_engine)
Alert
+target_id, severity
+message, event_type
+received_at
LogCollector
+collect_single(target_id) : dict
+collect_all() : dict
reads targets / writes CollectedLog
reuses cached or builds new
hwdiag + SCP path